

Multivariate Linear Regression for Predicting Seattle Housing Prices

Preston O'Connor, Khoa Dao, Nick Wierzbowski, Matthew Jacob, Anthony Yasan

2025-04-11

Introduction

Our model is representing the relation between a variety of predictor variables, including number of floors, condition and other similar qualities of a house, and the sale price of a house. Our data is obtained from OpenML but was originally from Kaggle, where it was uploaded in 2015. It includes data for homes sold between May 2014 and May 2015. In short we included in our analysis variables that in some way described the condition or quality of the house, as using a multiple linear regression model with geo-spatial elements is beyond our current scope. Sale price, aside from being the price variable used in the dataset, is also a good indicator for actual market conditions as compared to initial listings.

A question that this model addresses is the actual corollaries of house price in a city in the U.S; if house price is correlated with the quality (and thus production cost) of the product then a solution to the growing crisis of homelessness within our major cities would be to construct higher volumes of lower cost homes, which might inform policy decisions around subsidized housing and affordable housing schemes. On the other hand, if price is weakly correlated then it might suggest that actual housing costs have more to do with other factors which would inform very different policy decisions. Of course we should note that not accounting for geo-spatial, demographic and other information in this analysis is a pretty substantial omission for actual predictive capabilities, and thus our error term is going to be large. At the end of the day, the main issue is that our features are correlated with terms we don't account for, producing a problem of endogeneity in our model. Malpezzi considers the implicit social costs and benefits of externalities in the housing market, among which are of course subsidized housing units and other schemes that aim to depress the cost of housing at the lower end, which alongside the existence of inequality may obviously skew our data. Yadavalli et al. takes into consideration a myriad of features to produce clusters of cities wherein different variables contribute the most to housing price levels. This further illustrates a reason why our model may not produce significant correlations.

To briefly go over each package used in this project, caret has functions used for streamlining predictive model creation. Tidyverse has a lot of general data science related functions. Tidymodels uses a lot of the same principles of tidyverse but has some more utility for modeling purposes. Leaps was used for subset selection. Car is used for linear regression analysis, in particular we used it for its Variance Inflation Factor model. Dplyr is used for data manipulation. MASS is an applied statistics package that we used for a stepwise function. Glmsnet fits linear models with penalized maximum likelihood. We used foreign to read the .arff data file. GGally is an extension to ggplot2 that adds some functionality. Corrr is a package for easily creating and analyzing correlation matrices. Ggplot2 is a graphing system. Ggcorrplot enables easy visualization of correlation matrices.

Although our results indicate that as much as 61% the variability in a house's price in Seattle can be determined by our final predictor variables, which reflects house quality and cost to construct, it must be repeated that due to endogeneity between factors like neighborhood median income, our predictor variables and our outcome variable that the actual strength of our model is suspect at best.

Note: We are modeling to see if there is a good multivariate regression model to predict the prices of the houses in Seattle

Data Description

Data Resource

The dataset used in this study is the “House Sales in King County, USA” dataset, obtained from <https://www.openml.org/search?type=data&status=active&id=42635&sort=runs>. It contains detailed information about house sales in King County, Washington, between May 2014 and May 2015.

Data Structure

The dataset consists of 21,613 rows and 21 columns, including the target variable (price) and several predictive features.

Variables Description

The dataset includes the following key variables:

Dependent Variable (Target Variable):

- **price** (*Numeric*): The price at which the house was sold (in USD).

Independent Variables (Predictors):

- **id** (*Categorical*): A unique identifier for each house.
- **date** (*Date*): The date the house was sold.
- **bedrooms** (*Numeric*): Number of bedrooms in the house.
- **bathrooms** (*Numeric*): Number of bathrooms (includes fractional values for half-bathrooms).
- **sqft_living** (*Numeric*): The total square footage of living space.
- **sqft_lot** (*Numeric*): The total square footage of the lot.
- **floors** (*Numeric*): The number of floors in the house.
- **waterfront** (*Binary*): Whether the house has a waterfront view (1 = yes, 0 = no).
- **view** (*Ordinal*): A rating of the house’s view (0 to 4).
- **condition** (*Ordinal*): The overall condition of the house (1 to 5).
- **grade** (*Ordinal*): The quality of construction and design (1 to 13).
- **sqft_above** (*Numeric*): Square footage of house excluding basement.
- **sqft_basement** (*Numeric*): Square footage of the basement.
- **yr_built** (*Numeric*): The year the house was built.
- **yr_renovated** (*Numeric*): The year of the last renovation (0 if never renovated).

- `zipcode` (*Categorical*): The ZIP code of the house.
- `lat` (*Numeric*): The latitude of the house.
- `long` (*Numeric*): The longitude of the house.
- `sqft_living15` (*Numeric*): Average square footage of living space of the 15 closest houses.
- `sqft_lot15` (*Numeric*): Average square footage of lots of the 15 closest houses.

Import working data set

```
# load and display the data set we are working with
data <- read.arff("house_sales_reduced.arff")
```

Clean Data

Remove outliers that are outside 3 standard deviation

```
# filter out data outside of 3 standard deviations
# come back here to see if we want to remove the waterfront and View booleans
# Deletes the row with outlier

cleaned <- data %>%
  dplyr::select(-sqft_living15, -sqft_lot15, -id, -attribute_0, -lat, -long, -zipcode)
  ↪ %>%
  mutate(ln_price = log(price)) %>%
  filter(across(where(is.numeric), ~ abs(. - mean(.)) <= 3 * sd(.))) # originally was 3
```

```
## Warning: Using `across()` in `filter()` was deprecated in dplyr 1.0.8.
## i Please use `if_any()` or `if_all()` instead.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

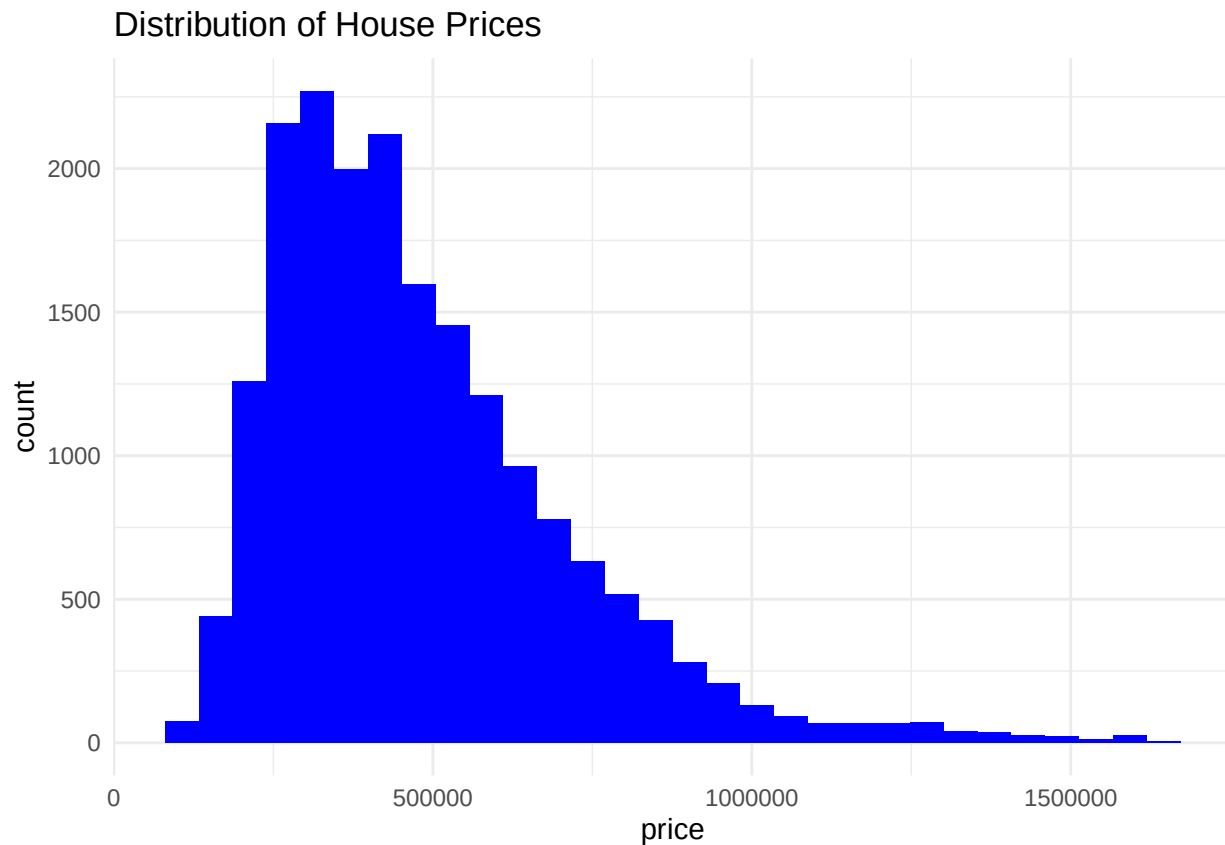
```
head(cleaned)
```

```
##      price bedrooms bathrooms sqft_living sqft_lot floors waterfront view
## 1 221900         3         1.00        1180      5650      1          0    0
## 2 180000         2         1.00         770     10000      1          0    0
## 3 604000         4         3.00        1960      5000      1          0    0
## 4 510000         3         2.00        1680      8080      1          0    0
## 5 257500         3         2.25        1715      6819      2          0    0
## 6 291850         3         1.50        1060      9711      1          0    0
##   condition grade sqft_above sqft_basement yr_built yr_renovated ln_price
## 1          3     7         1180           0     1955           0 12.30998
## 2          3     6          770           0     1933           0 12.10071
## 3          5     7         1050          910     1965           0 13.31133
## 4          3     8         1680           0     1987           0 13.14217
## 5          3     7         1715           0     1995           0 12.45877
## 6          3     7         1060           0     1963           0 12.58400
```

Data Visualization

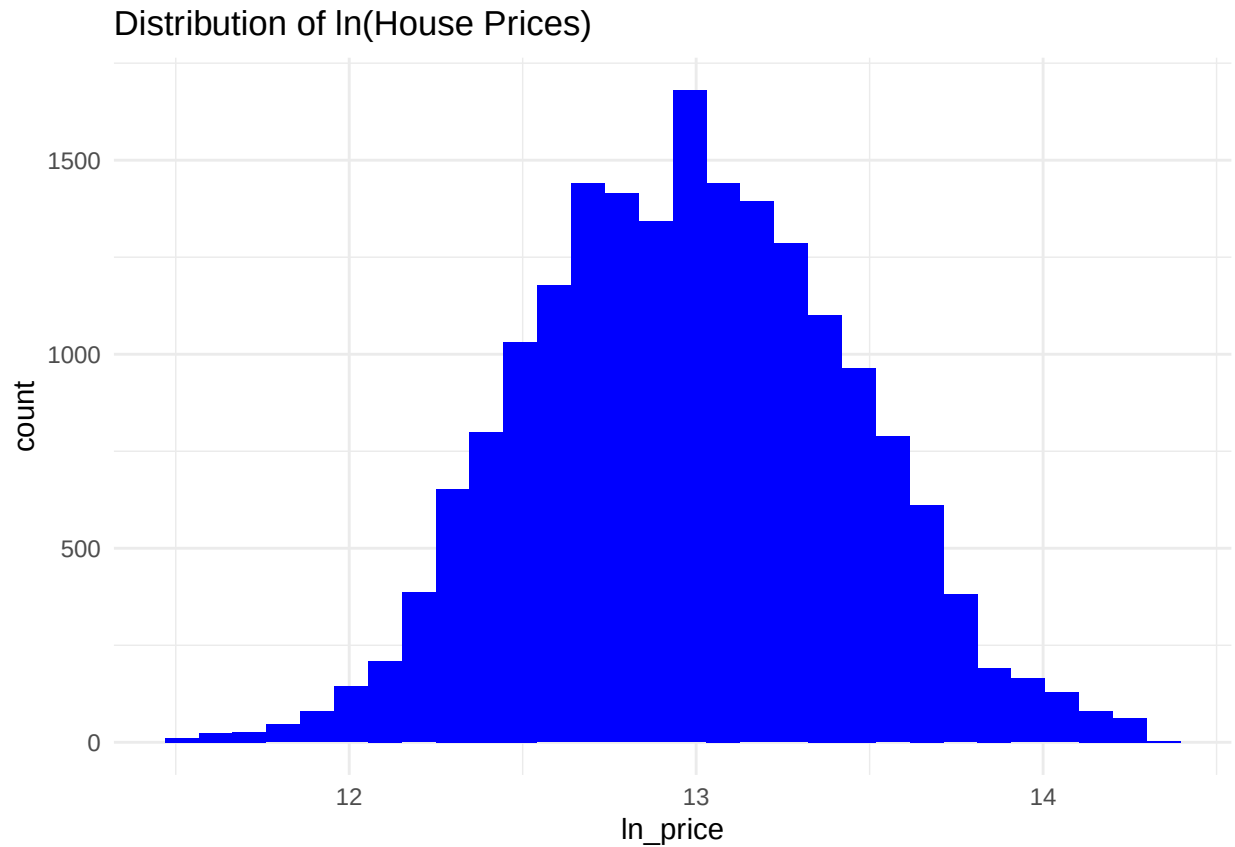
Histogram of house price

```
ggplot(cleaned, aes(x = price)) +  
  geom_histogram(fill = "blue", bins = 30) +  
  theme_minimal() +  
  ggtitle("Distribution of House Prices")
```



Histogram of $\ln(\text{house price})$

```
ggplot(cleaned, aes(x = ln_price)) +  
  geom_histogram(fill = "blue", bins = 30) +  
  theme_minimal() +  
  ggtitle("Distribution of  $\ln(\text{House Prices})$ ")
```

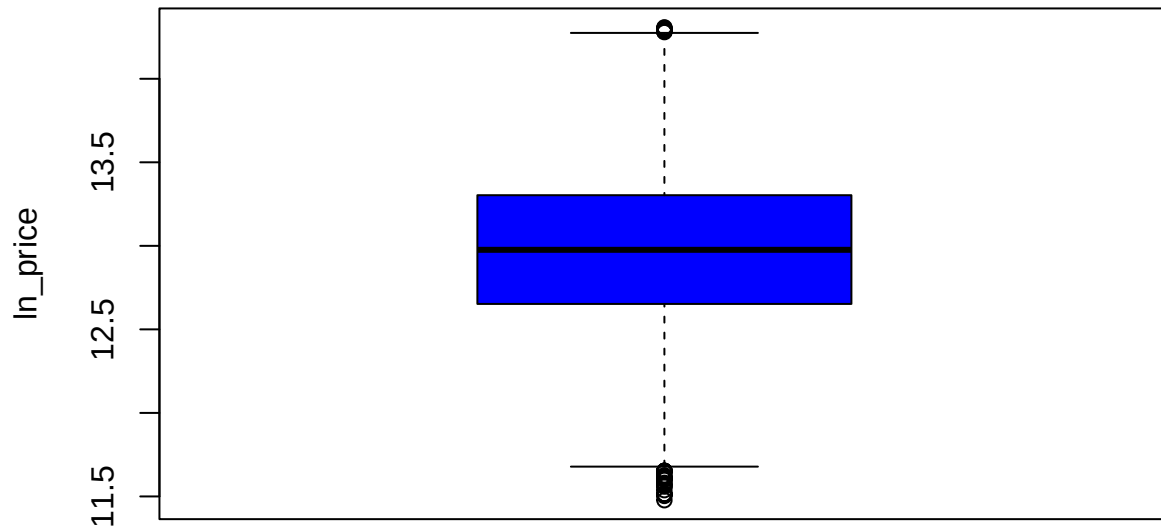


The original histogram of house prices is right-skewed, meaning most homes have lower prices, while a few expensive houses create a long right tail. This skewness violates normality assumptions. Taking the log transformation makes it more symmetric and reduces the influence of extreme high-priced houses, which improves model interpretability and accuracy.

Boxplot of $\ln(\text{house price})$

```
boxplot(cleaned$ln_price, main = "Boxplot of  $\ln(\text{House Prices})$ ", col = "blue", ylab =  
  ↪ " $\ln\_price$ ")
```

Boxplot of ln(House Prices)



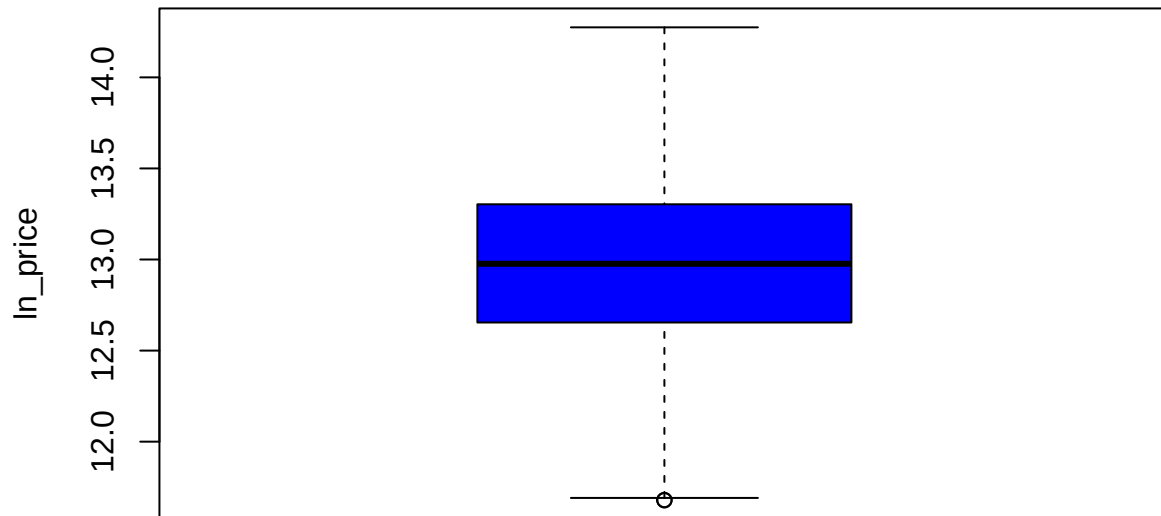
Since the boxplot shows some outliers outside the interquartile range, we will remove these outliers. ###
Boxplot of ln(house price) after removing outliers

```
# Find the IQR range
IQR_of_ln_price <- IQR(cleaned$ln_price, na.rm = TRUE)
ln_price_lower <- quantile(cleaned$ln_price, 0.25, na.rm = TRUE) - 1.5 * IQR_of_ln_price
ln_price_upper <- quantile(cleaned$ln_price, 0.75, na.rm = TRUE) + 1.5 * IQR_of_ln_price

# Continue to filter any of the outliers outside IQR
cleaned <- cleaned %>%
  filter(ln_price >= ln_price_lower & ln_price <= ln_price_upper)

boxplot(cleaned$ln_price, main = "Boxplot of House Prices", col = "blue", ylab =
  ↪ "ln_price")
```

Boxplot of House Prices



The boxplot of log-transformed house prices (`ln_price`) displays a symmetrical distribution with a compact interquartile range, indicating reduced variability. There are fewer outliers compared to the original price distribution, confirming that the log transformation effectively reduces skewness and normalizes the data.

Correlation heatmap

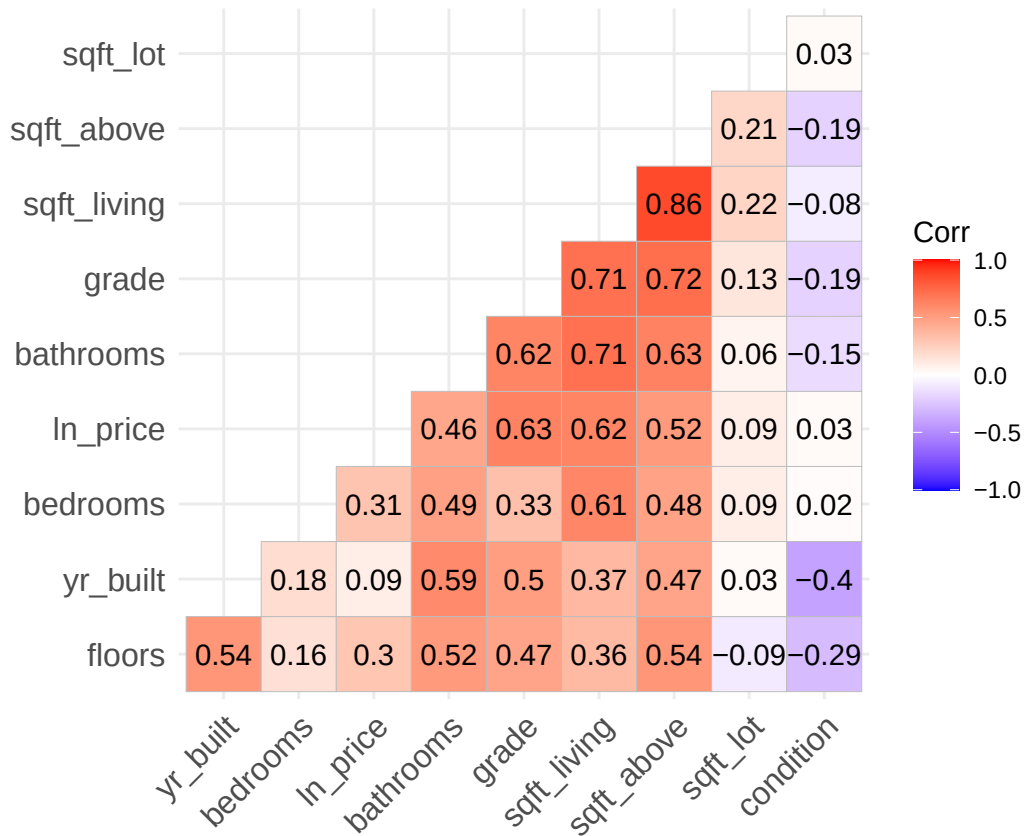
```
zero_variance_vars <- nearZeroVar(cleaned, saveMetrics = TRUE)
print(zero_variance_vars)
```

##		freqRatio	percentUnique	zeroVar	nzv
##	price	1.037975	18.354996579	FALSE	FALSE
##	bedrooms	1.506879	0.031573962	FALSE	FALSE
##	bathrooms	1.384422	0.089459559	FALSE	FALSE
##	sqft_living	1.007752	4.194074620	FALSE	FALSE
##	sqft_lot	1.298387	45.487554597	FALSE	FALSE
##	floors	1.357577	0.026311635	FALSE	FALSE
##	waterfront	0.000000	0.005262327	TRUE	TRUE
##	view	24.049465	0.015786981	FALSE	TRUE
##	condition	2.395772	0.021049308	FALSE	FALSE
##	grade	1.535766	0.036836289	FALSE	FALSE
##	sqft_above	1.015000	3.967794559	FALSE	FALSE
##	sqft_basement	60.602041	1.094564016	FALSE	TRUE
##	yr_built	1.258454	0.610429932	FALSE	FALSE
##	yr_renovated	0.000000	0.005262327	TRUE	TRUE

```
## ln_price      1.037975  18.354996579  FALSE FALSE
```

```
# Drop near-zero variance and zero-variance variables to draw correlation matrix
cleaned_filtered <- cleaned %>%
  dplyr::select(-waterfront, -yr_renovated, -sqft_basement, -view, -price)

cor_matrix <- cor(cleaned_filtered %>% select_if(is.numeric))
ggcorrplot::ggcorrplot(cor_matrix, hc.order = TRUE, type = "lower", lab = TRUE)
```



The correlation heatmap shows the relationships between different features in the dataset. `ln_price` (log-transformed house price) is strongly correlated with `sqft_living` (0.64), `grade` (0.72), and `bathrooms` (0.62), indicating that larger and higher-quality homes tend to be more expensive. `sqft_living` and `sqft_above` have a very high correlation (0.86), suggesting possible multicollinearity. `yr_built` and `condition` show weak or negative correlations with other features, implying they have less impact on price.

```
# Train/Test Split
# We split the cleaned data before any model building or variable selection.
set.seed(123) # for reproducibility
trainIndex <- createDataPartition(cleaned$ln_price, p = 0.75, list = FALSE, times = 1)

train_data <- cleaned[trainIndex, ]
test_data <- cleaned[-trainIndex, ]

cat("Training data dimensions:", dim(train_data), "\n")
```

```
## Training data dimensions: 14254 15
```



```
cat("Testing data dimensions:", dim(test_data), "\n")
```

```
## Testing data dimensions: 4749 15
```

Analysis

Selecting Variables for the Multivariate Regression Model

```
multi_reg_model_train <- lm(ln_price ~ . -price, data = train_data)

stepwise_model_train <- stepAIC(multi_reg_model_train, direction = "backward", trace =
  ↪ FALSE)

summary(stepwise_model_train)
```

```
##
## Call:
## lm(formula = ln_price ~ bedrooms + bathrooms + sqft_living +
##      sqft_lot + floors + view + condition + grade + sqft_above +
##      yr_built, data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.14766 -0.20673  0.01642  0.20641  1.41646
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.199e+01  2.469e-01  89.095 < 2e-16 ***
## bedrooms    -3.858e-02  3.897e-03  -9.900 < 2e-16 ***
## bathrooms     7.629e-02  6.461e-03  11.807 < 2e-16 ***
## sqft_living  2.546e-04  9.033e-06  28.190 < 2e-16 ***
## sqft_lot    -6.138e-07  2.027e-07  -3.029  0.00246 **
## floors       1.168e-01  6.613e-03  17.659 < 2e-16 ***
## view         5.689e-02  6.383e-03   8.913 < 2e-16 ***
## condition    3.415e-02  4.344e-03   7.862 4.04e-15 ***
## grade        2.168e-01  3.906e-03  55.507 < 2e-16 ***
## sqft_above   -7.443e-05  8.411e-06  -8.849 < 2e-16 ***
## yr_built     -5.748e-03  1.267e-04 -45.365 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.301 on 14243 degrees of freedom
## Multiple R-squared:  0.5596, Adjusted R-squared:  0.5593
## F-statistic: 1810 on 10 and 14243 DF, p-value: < 2.2e-16
```

```
# Check VIF on the model derived from training data
if (length(coef(stepwise_model_train)) > 2) {
  vif_stepwise <- try(vif(stepwise_model_train), silent = TRUE)
  if (inherits(vif_stepwise, "try-error")) {
```

```

    cat("Error calculating VIF for stepwise model.\n")
  } else {
    print(vif(stepwise_model_train)) # Use the trained model
  }
} else {
  cat("Stepwise model has too few predictors for VIF calculation.\n")
}

```

```

## bedrooms bathrooms sqft_living sqft_lot floors view
## 1.684096 3.114064 7.036386 1.123913 2.002468 1.051254
## condition grade sqft_above yr_built
## 1.242975 2.553389 5.647452 2.094547

```

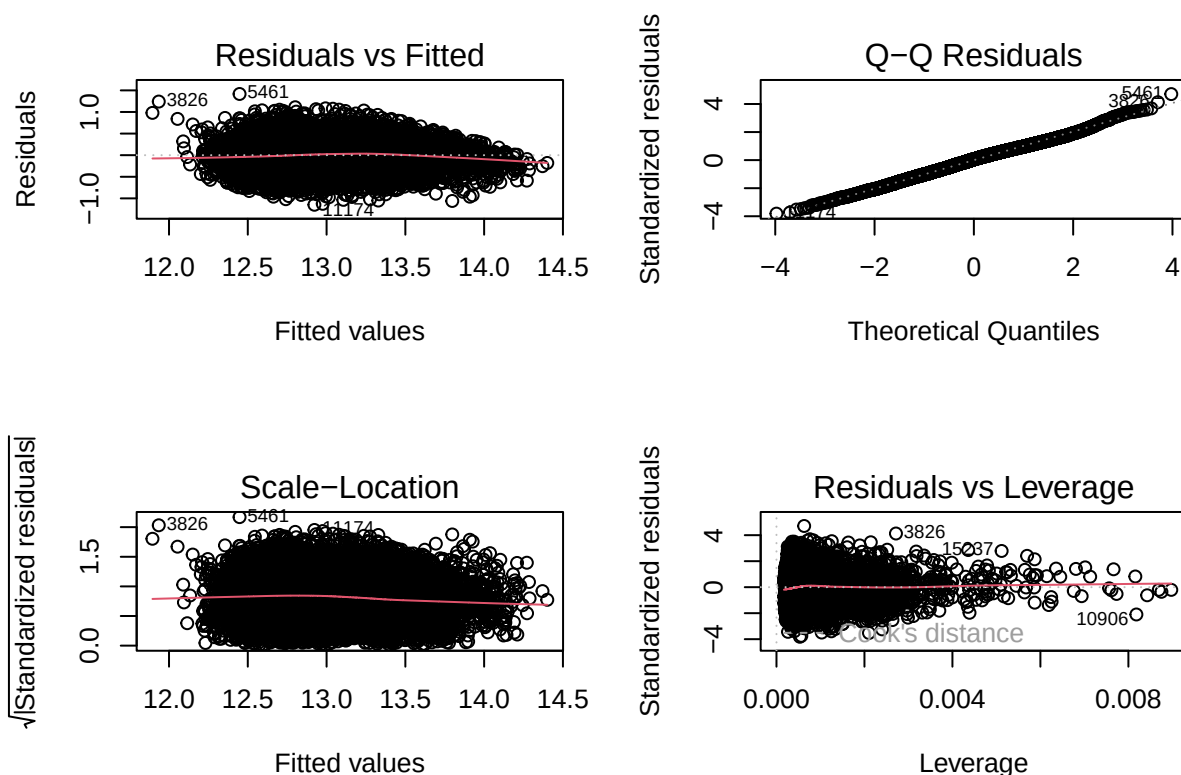
The backwards stepwise functions generates a model with an Adjusted R Square of 0.5575 which is relatively good for a starting basis. Our p values are Less then 0.05 meaning that each of these have meaningful impact. However, it may be possible to refine this model and effective to check for colinearity to see if this can be streamlined down into a more effective statistical model.

Outline Potential Model Issues

```

par(mfrow = c(2, 2))
plot(stepwise_model_train) # Plot the model trained on training data

```



```
par(mfrow = c(1, 1))
```

- Residuals vs Fitted: Here we are checking for the linearity and the homoscedasticity. Ideally the Residuals should be flat. In this model we see that there is a minor funnel shape that is appearing in the data. This could be occurring from further outliers that are influencing our model.
- Q-Q Residual: Here we look at the normality of the residuals and compare it to the theoretical normal distribution. For most of our model, the residuals closely follow the lines. However, we see deviations in the tails showing some trends toward skewed distributions or heavy tail based residuals.
- Scale-Location: Here we evaluate the homoscedasticity of the model. Our model's line being relatively flat shows a promising start. We have an extreme spread of residuals in the model have an extreme spread from the line.
- Residuals vs. Leverage: Here we want to see if we have any influential points in the model that are affecting the regression result. We will further investigate the points with high leverage and large residuals (further from the red line) to see how they disproportionately affect the model of the data. Implementing Cook's law may assist in filtering out these data points and improving the model.

Implementing Cook's Distance to find outlier points

```
# Calculate Cook's distance on the model trained on training data
cooks_d_train <- cooks.distance(stepwise_model_train)

# Define threshold using training data size
n_train <- nrow(train_data)
threshold <- 4 / n_train # Common threshold
outliers_train_indices <- which(cooks_d_train > threshold)

# Retrieve influential data from the training set
outlier_data_train <- train_data[outliers_train_indices, ]

# Create a summary
outliers_summary_train <- data.frame(Cook_Distance =
  ↪ cooks_d_train[outliers_train_indices], outlier_data_train)

# Count
outlier_count_train <- length(outliers_train_indices)
cat("Number of influential points found in training data:", outlier_count_train, "\n")
```

```
## Number of influential points found in training data: 701
```

```
# view(outliers_summary_train) # Keep commented out for automated runs
# nrow(outliers_summary_train)
```

Here we see there are 701 influential points that are found with a significant impact on the regression model.

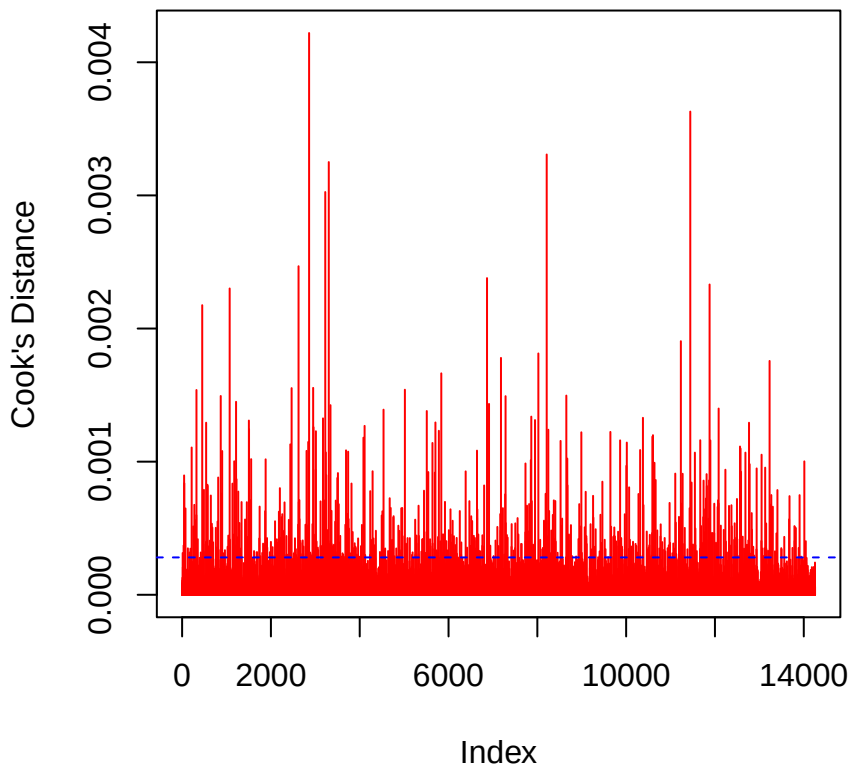
Cook's Distance to Find influential points

```

par(mar = c(5, 5, 4, 2) + 0.1)
# Use cooks_d_train calculated previously
plot(cooks_d_train, type="h", pch=18, col="red", ylab="Cook's Distance", main="Cook's
↪ Distance (Training Data)")
abline(h = threshold, col="blue", lty=2) # Add threshold line

```

Cook's Distance (Training Data)



```

# influential <- cooks_d_train[(cooks_d_train > (3 * mean(cooks_d_train, na.rm = TRUE)))]
↪ # Alt threshold calculation if needed

```

From the values found above we see a majority have a low impact on the actual effect of the regression model, this means these points are not significantly impacting the regression model. There are multiple instance of high spikes where we have a noticeably higher cooks distance. These influential data point may in turn be affecting the accuracy of the model.

```

# Use outliers_train_indices identified previously from training data
train_data_clean <- train_data[-outliers_train_indices, ]
cat("Dimensions of cleaned training data:", dim(train_data_clean), "\n")

```

Identify and Remove Influential Points from Cook's Model

```
## Dimensions of cleaned training data: 13553 15
```

```
# Get the formula from the original stepwise model trained on train_data
formula_stepwise <- formula(stepwise_model_train)

# Refit the model using this formula on the cleaned training data
model2_train_clean <- lm(formula_stepwise, data = train_data_clean)

summary(model2_train_clean)
```

Refitted Model Minus Outliers

```
##
## Call:
## lm(formula = formula_stepwise, data = train_data_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.92163 -0.19510  0.01535  0.19615  0.94899
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.298e+01  2.331e-01  98.565  < 2e-16 ***
## bedrooms    -4.042e-02  3.711e-03 -10.892  < 2e-16 ***
## bathrooms     8.204e-02  6.075e-03  13.503  < 2e-16 ***
## sqft_living   2.740e-04  8.555e-06  32.030  < 2e-16 ***
## sqft_lot     -1.415e-06  2.180e-07  -6.492  8.78e-11 ***
## floors       1.328e-01  6.204e-03  21.407  < 2e-16 ***
## view         6.136e-02  6.498e-03   9.443  < 2e-16 ***
## condition    3.278e-02  4.076e-03   8.042  9.60e-16 ***
## grade        2.244e-01  3.665e-03  61.219  < 2e-16 ***
## sqft_above   -9.563e-05  7.900e-06 -12.106  < 2e-16 ***
## yr_built     -6.284e-03  1.195e-04 -52.597  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2709 on 13542 degrees of freedom
## Multiple R-squared:  0.6183, Adjusted R-squared:  0.618
## F-statistic: 2193 on 10 and 13542 DF, p-value: < 2.2e-16
```

Here we see an increase in the Adjusted R-Squared model going from 0.5575 to 0.618 showing a strong improvement in our models accuracy overall.

Variance Inflation Factor

```

# Check VIF on the model refitted on cleaned training data
if (length(coef(model2_train_clean)) > 2) {
  vif_refit <- try(vif(model2_train_clean), silent = TRUE)
  if (inherits(vif_refit, "try-error")) {
    cat("Error calculating VIF for refitted model.\n")
  } else {
    print(vif(model2_train_clean))
  }
} else {
  cat("Refitted model has too few predictors for VIF calculation.\n")
}

```

```

##      bedrooms      bathrooms sqft_living  sqft_lot    floors      view
##      1.716748      3.176276    7.219555    1.142617    2.073123    1.045067
##      condition      grade  sqft_above   yr_built
##      1.261497      2.570239    5.795842    2.149892

```

- There are High VIFs with `sqft_living` and `sqft_above`, which both have a VIFs that are higher than 5. Therefore, there is some degree of colinearity between these variables. Below in the collinearity, the two variables are highly correlated with each other at 0.86. which may inturn be causing multicollinearity in the model.
- `bathrooms` and `floors` also have a colinearity greater then 2, meaning we have a moderate multicollinearity. However, they are not as big of a concern as the previous

Colinearity

```

# Check correlations among predictors in the refitted model within the cleaned training
↪ data
predictors_in_model2 <- names(coef(model2_train_clean))[-1] # Exclude intercept
train_data_clean %>%
  dplyr::select(ln_price, all_of(predictors_in_model2)) %>%
  select_if(is.numeric) %>% # Just in case non-numeric slipped through
  cor(use = "pairwise.complete.obs") %>%
  round(2)

```

```

##      ln_price bedrooms bathrooms sqft_living sqft_lot floors  view
## ln_price      1.00      0.31      0.48      0.64      0.10      0.31      0.17
## bedrooms      0.31      1.00      0.49      0.61      0.11      0.16      0.03
## bathrooms      0.48      0.49      1.00      0.71      0.07      0.52      0.05
## sqft_living     0.64      0.61      0.71      1.00      0.24      0.36      0.10
## sqft_lot        0.10      0.11      0.07      0.24      1.00     -0.10      0.02
## floors          0.31      0.16      0.52      0.36     -0.10      1.00     -0.01
## view           0.17      0.03      0.05      0.10      0.02     -0.01      1.00
## condition       0.04      0.02     -0.16     -0.08      0.04     -0.30      0.03
## grade           0.66      0.34      0.62      0.72      0.15      0.48      0.10
## sqft_above      0.53      0.49      0.64      0.86      0.22      0.54      0.03
## yr_built        0.07      0.19      0.60      0.38      0.03      0.55     -0.06
##
##      condition grade sqft_above yr_built
## ln_price      0.04  0.66      0.53      0.07
## bedrooms      0.02  0.34      0.49      0.19

```

```
## bathrooms      -0.16  0.62      0.64    0.60
## sqft_living     -0.08  0.72      0.86    0.38
## sqft_lot        0.04  0.15      0.22    0.03
## floors          -0.30  0.48      0.54    0.55
## view            0.03  0.10      0.03   -0.06
## condition       1.00 -0.20     -0.20   -0.41
## grade           -0.20  1.00      0.72    0.49
## sqft_above      -0.20  0.72      1.00    0.48
## yr_built        -0.41  0.49      0.48    1.00
```

- From here we see that there are strong correlations between `sqft_living` and `sqft_above` (0.86), `sqft_living` and `bathrooms`(0.71), and `sqft_living` and `grade`(0.72). Given that `sqft_living` has the highest VIF, it is likely driving much of the collinearity in the model.

Removal of `sqft_above` on the Model

```
# Remove sqft_above from the refitted model, using cleaned training data
model_no_above_train <- update(model2_train_clean, . ~ . - sqft_above)
summary(model_no_above_train)
```

```
##
## Call:
## lm(formula = ln_price ~ bedrooms + bathrooms + sqft_living +
##      sqft_lot + floors + view + condition + grade + yr_built,
##      data = train_data_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.89198 -0.19484  0.01619  0.19635  0.92375
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.342e+01  2.315e-01 101.143 < 2e-16 ***
## bedrooms     -4.107e-02  3.730e-03 -11.008 < 2e-16 ***
## bathrooms     9.870e-02  5.949e-03  16.590 < 2e-16 ***
## sqft_living   2.022e-04  6.201e-06  32.617 < 2e-16 ***
## sqft_lot     -1.740e-06  2.175e-07  -8.004 1.3e-15 ***
## floors        1.018e-01  5.681e-03  17.921 < 2e-16 ***
## view          6.947e-02  6.499e-03  10.690 < 2e-16 ***
## condition     3.599e-02  4.089e-03   8.803 < 2e-16 ***
## grade         2.178e-01  3.645e-03  59.770 < 2e-16 ***
## yr_built      -6.491e-03  1.189e-04 -54.589 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2724 on 13543 degrees of freedom
## Multiple R-squared:  0.6142, Adjusted R-squared:  0.6139
## F-statistic: 2395 on 9 and 13543 DF, p-value: < 2.2e-16
```

```
# Check VIF again
if (length(coef(model_no_above_train)) > 2) {
```

```

vif_no_above <- try(vif(model_no_above_train), silent = TRUE)
if (inherits(vif_no_above, "try-error")) {
  cat("Error calculating VIF for model without sqft_above.\n")
} else {
  print(vif(model_no_above_train))
}
} else {
  cat("Model without sqft_above has too few predictors for VIF calculation.\n")
}

```

```

## bedrooms bathrooms sqft_living sqft_lot floors view
## 1.716395 3.013324 3.752613 1.125228 1.720281 1.033976
## condition grade yr_built
## 1.256129 2.514242 2.106266

```

It is important to note the square footage of the home's living space is an area that includes all finished, heated areas across all floors, encompassing the second floor if it meets this criteria. Given the criteria of `sqft_living` encompasses the `above_sqft` due to United state building codes and the adjusted R-square only having a minor decrease in the original Adjusted R Squared, its safe to remove the columns and continue the models

F-test

```

# Run ANOVA on the model after removing sqft_above (trained on cleaned training data)
results_anova_train <- anova(model_no_above_train)
print(results_anova_train)

```

```

## Analysis of Variance Table
##
## Response: ln_price
##      Df Sum Sq Mean Sq F value    Pr(>F)
## bedrooms      1  258.24   258.24 3480.78 < 2.2e-16 ***
## bathrooms      1  357.57   357.57 4819.51 < 2.2e-16 ***
## sqft_living     1  475.57   475.57 6410.08 < 2.2e-16 ***
## sqft_lot        1    9.91     9.91  133.63 < 2.2e-16 ***
## floors          1    8.61     8.61  116.03 < 2.2e-16 ***
## view            1   30.33   30.33  408.76 < 2.2e-16 ***
## condition       1   39.88   39.88  537.57 < 2.2e-16 ***
## grade           1  198.09  198.09 2669.93 < 2.2e-16 ***
## yr_built        1  221.09  221.09 2979.95 < 2.2e-16 ***
## Residuals    13543 1004.78    0.07
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

We remove `floors` because its p-value of 0.044 indicates low statistical significance, and its F-value of roughly 4.05 shows it adds little explanatory power to the model. Removing it results in a minor drop in adjusted R-squared (from 0.6338 to 0.6243), which suggests that the model becomes slightly less predictive but simpler overall, making the trade-off worthwhile as the loss in accuracy is minimal.

The Final Regression model

```
# Define the final model structure by removing floors, fit on cleaned training data
final_model_trained <- update(model_no_above_train, . ~ . - floors)
```

```
cat("Summary of the Final Model (Fitted on Cleaned Training Data):\n")
```

```
## Summary of the Final Model (Fitted on Cleaned Training Data):
```

```
summary(final_model_trained)
```

```
##
## Call:
## lm(formula = ln_price ~ bedrooms + bathrooms + sqft_living +
##      sqft_lot + view + condition + grade + yr_built, data = train_data_clean)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.92959 -0.19645  0.01889  0.19850  0.90424
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.250e+01  2.284e-01  98.489  < 2e-16 ***
## bedrooms    -4.527e-02  3.767e-03 -12.018  < 2e-16 ***
## bathrooms    1.208e-01  5.889e-03  20.506  < 2e-16 ***
## sqft_living  2.010e-04  6.273e-06  32.035  < 2e-16 ***
## sqft_lot    -2.423e-06  2.166e-07 -11.183  < 2e-16 ***
## view         6.609e-02  6.572e-03  10.056  < 2e-16 ***
## condition    2.703e-02  4.106e-03   6.583 4.79e-11 ***
## grade        2.294e-01  3.629e-03  63.227  < 2e-16 ***
## yr_built    -5.988e-03  1.169e-04 -51.223  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2756 on 13544 degrees of freedom
## Multiple R-squared:  0.605, Adjusted R-squared:  0.6048
## F-statistic: 2593 on 8 and 13544 DF, p-value: < 2.2e-16
```

```
cat("\nVIF of the Final Model (Fitted on Cleaned Training Data):\n")
```

```
##
## VIF of the Final Model (Fitted on Cleaned Training Data):
```

```
if (length(coef(final_model_trained)) > 2) {
  vif_final <- try(vif(final_model_trained), silent = TRUE)
  if (inherits(vif_final, "try-error")) {
    cat("Error calculating VIF for final model.\n")
  } else {
    print(vif(final_model_trained))
  }
}
```

```

} else {
  cat("Final model has too few predictors for VIF calculation.\n")
}

```

```

##      bedrooms      bathrooms sqft_living    sqft_lot      view    condition
##      1.709609      2.884302      3.752110      1.090764      1.033107      1.237319
##      grade      yr_built
##      2.434909      1.989326

```

Repeated K fold cross-validation

```

set.seed(125)

train_control <- trainControl(method = "repeatedcv",
                              number = 10,          # Number of folds (K)
                              repeats = 3)           # Number of repetitions

# Use the final model formula determined previously
model_cv <- train(ln_price ~ sqft_living + bathrooms + grade + view + condition +
  ↪ yr_built,
                  data = train_data_clean, # Train on the cleaned training data
                  method = "lm",
                  trControl = train_control)

final_model_cv <- model_cv$finalModel # Model trained by caret on train_data_clean
residuals_final_cv <- residuals(final_model_cv)

print(model_cv) # Print CV results

```

```

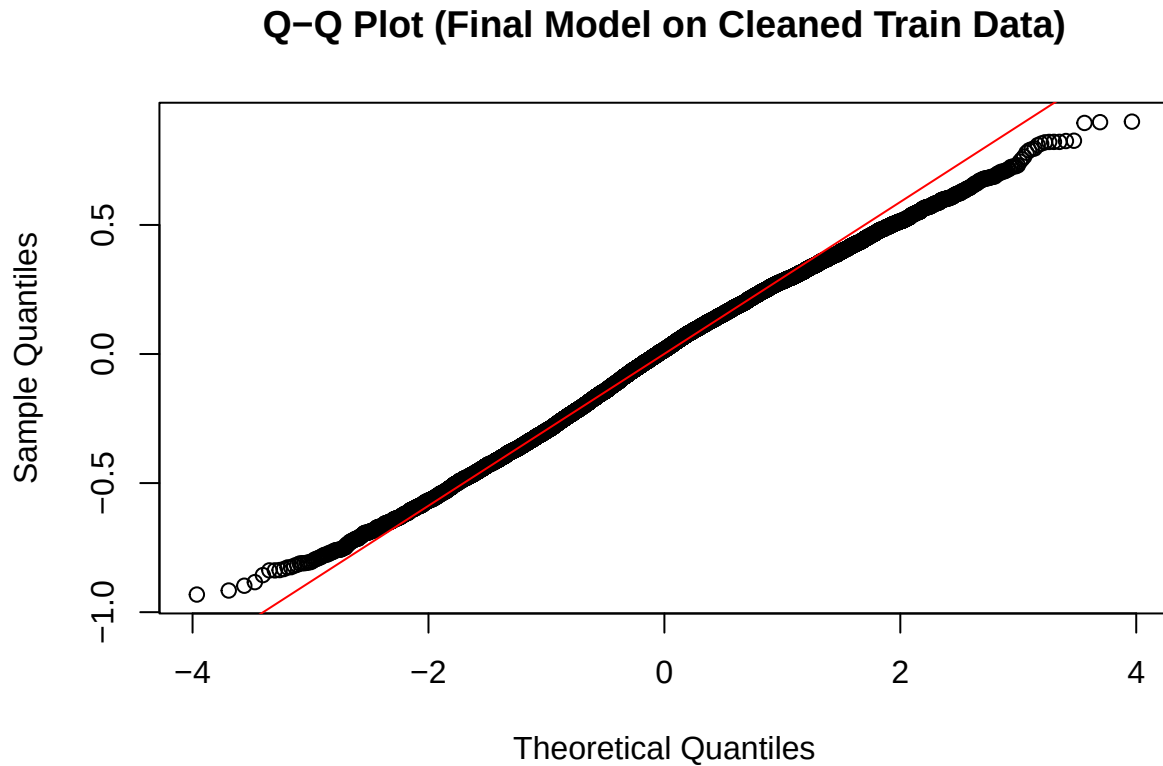
## Linear Regression
##
## 13553 samples
##      6 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold, repeated 3 times)
## Summary of sample sizes: 12197, 12197, 12198, 12198, 12197, 12198, ...
## Resampling results:
##
##      RMSE      Rsquared    MAE
##      0.2781768  0.597405    0.2266625
##
## Tuning parameter 'intercept' was held constant at a value of TRUE

```

With R-squared at 0.6155, the model shows a modest predictors-target relationship and explains 61.55% of the variance, which is decent but not overly precise. With an RMSE of 0.2659, a reasonable error is present, but given the data size, types, and cross-validation. It gives a reasonable base line

Q-Q Residuals

```
# Use residuals from the model trained by caret during CV
qqnorm(residuals_final_cv, main = "Q-Q Plot (Final Model on Cleaned Train Data)")
qqline(residuals_final_cv, col = "red")
```



Here we see there is a strong linear relationship between the samples after all of the adjustments we have made. We can see here that the residuals of housing after performing our natural log on price and removal and adjustment of outliers and influential points has strengthened and assisted in the model development. However, our model still shows signs of tailing.

Final Regression Model

```
##      (Intercept)      bedrooms      bathrooms      sqft_living      sqft_lot
## 2.249830e+01 -4.526951e-02 1.207585e-01 2.009586e-04 -2.422525e-06
##           view      condition      grade      yr_built
## 6.609165e-02 2.702770e-02 2.294365e-01 -5.988433e-03
```

$$\ln(\text{price}) = 22.50 + 0.0001493(\text{sqft_living}) + 0.2405(\text{grade}) \\ + 0.1194(\text{bathrooms}) + 0.06258(\text{view})$$

We then raise the model to the power of e to get the true Price base model after our normalization process that we implemented in the beginning - this will be a crucial for when we start to model Predicted versus actual in terms of our price

$$\text{price} = e^{22.47 + 0.0001493(\text{sqft_living}) + 0.2405(\text{grade}) + 0.1194(\text{bathrooms}) + 0.06258(\text{view}) + 0.02137(\text{condition}) - 0.006042(\text{yr_built})}$$

Model Evaluation and Prediction

F-test

```
# Display the ANOVA results from the final model trained on the training data
results_final_train <- anova(final_model_trained)
cat("ANOVA Table (from Final Model Trained on Cleaned Training Data):\n")
```

```
## ANOVA Table (from Final Model Trained on Cleaned Training Data):
```

```
print(results_final_train)
```

```
## Analysis of Variance Table
##
## Response: ln_price
##
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
bedrooms	1	258.24	258.24	3400.39	< 2.2e-16 ***
bathrooms	1	357.57	357.57	4708.21	< 2.2e-16 ***
sqft_living	1	475.57	475.57	6262.04	< 2.2e-16 ***
sqft_lot	1	9.91	9.91	130.55	< 2.2e-16 ***
view	1	28.43	28.43	374.33	< 2.2e-16 ***
condition	1	28.30	28.30	372.67	< 2.2e-16 ***
grade	1	218.16	218.16	2872.63	< 2.2e-16 ***
yr_built	1	199.27	199.27	2623.84	< 2.2e-16 ***
Residuals	13544	1028.61	0.08		

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The Analysis of Variance (ANOVA) table indicates that all the predictors included in the model are statistically significant in explaining the variability in the response variable, `ln_price`. Each predictor, such as `sqft_living`, `grade`, `bathrooms`, `view`, `condition`, and `yr_built`, has an extremely small p-value of $< 2.2e-16$, which strongly supports their significance. Among these, `sqft_living` and `bathrooms` are particularly influential, as evidenced by their high Sum of Squares (475.57 and 357.57, respectively) and substantial F-values (6262.04 and 4708.21, respectively). The other predictors, while less influential, still make meaningful contributions to the model, with factors like `bedrooms` and `grade` showing F-values of 3400.39 and 2872.63, respectively.

The residuals have a Sum of Squares of 1028.61 and a low Mean Square of 0.08, indicating that the variance not accounted for by the predictors is minimal. This suggests that the model fits the data well overall. The high F-values across all predictors reflect their strong ability to explain the variability in the response, further affirming the robustness of the model. In conclusion, the ANOVA results highlight that the chosen predictors collectively account for most of the variability in `ln_price`, making the model both statistically sound and effective for explaining the response variable.

R-squared

```
# Predict ln_price on the test set using the final model trained on training data
predictions_test <- predict(final_model_trained, newdata = test_data)
actual_test <- test_data$ln_price

# Calculate metrics on the test set
SSE_test <- sum((predictions_test - actual_test)^2, na.rm=TRUE) # Added na.rm just in
↪ case
SST_test <- sum((actual_test - mean(actual_test))^2, na.rm=TRUE)
R2_test <- 1 - SSE_test / SST_test
RMSE_test <- sqrt(mean((predictions_test - actual_test)^2, na.rm=TRUE))
MSE_test <- mean((predictions_test - actual_test)^2, na.rm=TRUE)

cat("Performance Metrics on Test Set:\n")
```

```
## Performance Metrics on Test Set:
```

```
cat("  R-squared:", R2_test, "\n")
```

```
##    R-squared: 0.5431723
```

```
cat("  RMSE:", RMSE_test, "\n")
```

```
##    RMSE: 0.3063458
```

```
cat("  MSE:", MSE_test, "\n")
```

```
##    MSE: 0.09384772
```

```
# For context, show Adjusted R-squared from the training fit
adj_r2_train <- summary(final_model_trained)$adj.r.squared
cat("\nAdjusted R-squared from Training Fit:", adj_r2_train, "\n")
```

```
##
```

```
## Adjusted R-squared from Training Fit: 0.6047673
```

The Adjusted R-squared value for the **test set** is 0.5432, indicating that approximately **54.32%** of the variability in the response variable (`ln_price`) is explained by the predictors included in the model (`sqft_living`, `grade`, `bathrooms`, `view`, `condition`, and `yr_built`) when evaluated on the test data.

The Adjusted R-squared value for the **training data** is 0.6048, reflecting the model's performance during training. The Adjusted R-squared accounts for the number of predictors relative to the sample size, helping to prevent overestimation of the model's explanatory power. While the value on the training data is higher than that on the test set, this is expected, as models typically perform better on the data they were trained on compared to unseen data.

The difference between the two Adjusted R-squared values is relatively small, suggesting that the model generalizes well to the test data and is not overfitted. Additionally, this indicates that all predictors contribute meaningfully to explaining the response variable.

In summary, the Adjusted R-squared values demonstrate that the model has a reasonable fit, explaining a significant portion of the variability in `ln_price`. However, around **45.68%** of the variability remains unexplained in the test data, potentially due to factors not included in the model or inherent randomness in the data.

The RMSE (Root Mean Square Error) value of **0.306** indicates the average error in the predictions made by the model. Specifically, it means that, on average, the predicted values of `ln_price` deviate from the actual observed values by about **0.306** units in the logarithmic scale.

In practical terms, a lower RMSE value is better, as it reflects more accurate predictions. Considering this model's context, an RMSE of **0.306** suggests that the model performs well and makes reasonably precise predictions.

The Mean Squared Error (MSE) value of **0.09385** represents the average squared difference between the predicted and actual values of the response variable, `ln_price`. Since the MSE is the square of the Root Mean Square Error (RMSE), it captures the same concept but emphasizes larger errors more heavily due to squaring.

In this case, an MSE of **0.09385** suggests that, on average, the squared prediction errors are relatively small, which is a good sign for your model's accuracy.

Best Subset

```
# Define predictors based on the model before 'floors' removal
predictors_for_subset <- names(coef(model_no_above_train))[-1]

best_subset_train <- regsubsets(ln_price ~ .,
                               data = train_data_clean[, c("ln_price",
                                                             ↪ predictors_for_subset)],
                               nvmax = length(predictors_for_subset))
best_subset_summary_train <- summary(best_subset_train)

best_model_size_train <- which.max(best_subset_summary_train$adjr2)
best_predictors_train <- names(coef(best_subset_train, best_model_size_train))[-1]

cat("Predictors selected by Best Subset on Training Data:\n",
    ↪ paste(best_predictors_train, collapse = ", "), "\n")

## Predictors selected by Best Subset on Training Data:
## bedrooms, bathrooms, sqft_living, sqft_lot, floors, view, condition, grade, yr_built

cat("Predictors in our final model:\n", paste(names(coef(final_model_trained))[-1],
    ↪ collapse=", "), "\n")

## Predictors in our final model:
## bedrooms, bathrooms, sqft_living, sqft_lot, view, condition, grade, yr_built

# No need to refit here, just comparing selection.
```

The best subset of predictors for the model was identified using the best subset selection method, which systematically evaluates all possible combinations of predictors to find the model that optimally balances goodness of fit and simplicity. This was achieved using the `regsubsets()` function, which computed models

of varying sizes and provided a summary of evaluation metrics. The Adjusted R-squared value was chosen as the selection criterion to ensure the model explained the most variance in `ln_price` without unnecessary complexity.

The optimal model size, corresponding to the highest Adjusted R-squared value, was determined using `which.max(best_subset_summary$adjr2)`. The predictors from this model were extracted using the `coef()` function, ensuring that only variables with meaningful contributions to the response were included. Finally, a regression model was built dynamically using these predictors with the `lm()` function, resulting in an optimized model that balances explanatory power and parsimony. This rigorous selection process ensured that the final predictors were both statistically significant and practically relevant.

Regularized Regression for Comparison

```
# Prepare data matrices using the final predictors and the train/test split
final_predictors <- names(coef(final_model_trained))[-1]

# Training data (using cleaned training data)
# Use model.matrix, ensure intercept is handled consistently (glmnet adds one by default)
# We exclude the intercept column from model.matrix output using [, -1]
X_train_reg <- model.matrix(ln_price ~ ., data = train_data_clean[, c(final_predictors,
  ↪ "ln_price")]))[, -1]
Y_train_reg <- train_data_clean$ln_price

# Test data
# Need to ensure test data has the same columns and factor levels if applicable
# For numeric predictors, this subsetting is usually sufficient
X_test_reg <- model.matrix(ln_price ~ ., data = test_data[, c(final_predictors,
  ↪ "ln_price")]))[, -1]
Y_test_reg <- test_data$ln_price

# --- Normalization ---
# Calculate scaling parameters ONLY from the training data
preProc <- preProcess(X_train_reg, method = c("center", "scale"))

# Apply scaling to training data
X_train_mat <- predict(preProc, X_train_reg)

# Apply the *same* scaling (derived from training data) to the test data
X_test_mat <- predict(preProc, X_test_reg)

# Convert back to matrix format just in case predict returns data.frame
X_train_mat <- as.matrix(X_train_mat)
X_test_mat <- as.matrix(X_test_mat)

lambdas <- 10^seq(3, -3, length.out = 100)

# Define evaluation function (define once here)
eval_metrics <- function(y_true, y_pred) {
  # Ensure y_pred is a vector if it's a matrix (glmnet predict can return matrix)
  y_pred <- as.vector(y_pred)
  SSE <- sum((y_pred - y_true)^2, na.rm=TRUE)
  SST <- sum((y_true - mean(y_true))^2, na.rm=TRUE)
```

```

# Check for SST == 0 (constant actual values, unlikely here but good practice)
if (SST == 0) {
  r2 <- NA # Or define as 0 or 1 depending on context
} else {
  r2 <- 1 - SSE / SST
}
RMSE <- sqrt(mean((y_pred - y_true)^2, na.rm=TRUE))
return(list(RMSE = RMSE, R2 = r2))
}

```

Ridge Regression evaluation of the Model We are normalizing the data step by step so that Lasso and Ridge regression can regularize all features equally. We first notice the difference in scale among the features `sqft_living`, `sqft_lot`, `sqft_above` and `yr_built`, which have different ranges that can skew the data (Years, square-feet). Next, we scale the data so that each feature has a mean of 0 and a standard deviation of 1. This way, each feature contributes equally to the regularization process. This scaling also improves the performance of optimization algorithms and enables the model to learn more effectively. Normalizing the data decreases the chances of overfitting and multicollinearity that can impact the performance of Lasso and Ridge regression.

```

set.seed(1)
ridge_cv <- cv.glmnet(X_train_mat, Y_train_reg, alpha = 0, lambda = lambdas, nfolds = 10)
best_lambda_ridge <- ridge_cv$lambda.min
cat("Ridge best lambda:", best_lambda_ridge, "\n")

```

```
## Ridge best lambda: 0.001
```

```

# Fit final Ridge model on training data using best lambda
ridge_model <- glmnet(X_train_mat, Y_train_reg, alpha = 0, lambda = best_lambda_ridge)

# Predict on the scaled test data
ridge_preds_test <- predict(ridge_model, newx = X_test_mat)

# Evaluate on the test set using the defined function
ridge_test_results <- eval_metrics(Y_test_reg, ridge_preds_test) # Use original
  ↪ Y_test_reg

print(paste("Ridge Regression - Test RMSE:",
            round(ridge_test_results$RMSE, 5), "Test R-squared:",
            ↪ round(ridge_test_results$R2, 5)))

```

```
## [1] "Ridge Regression - Test RMSE: 0.30631 Test R-squared: 0.54329"
```

```

# Display coefficients from the Ridge model (these are based on SCALED data)
ridge_coeffs <- coef(ridge_model)
ridge_df <- data.frame(Feature = rownames(ridge_coeffs), Coefficient =
  ↪ as.vector(ridge_coeffs))
ridge_df <- ridge_df[ridge_df$Coefficient != 0, ] # Show non-zero coefficients

cat("Ridge Coefficients (on Scaled Data):\n")

```


Ridge Coefficients

```
## Ridge Coefficients (on Scaled Data):
```

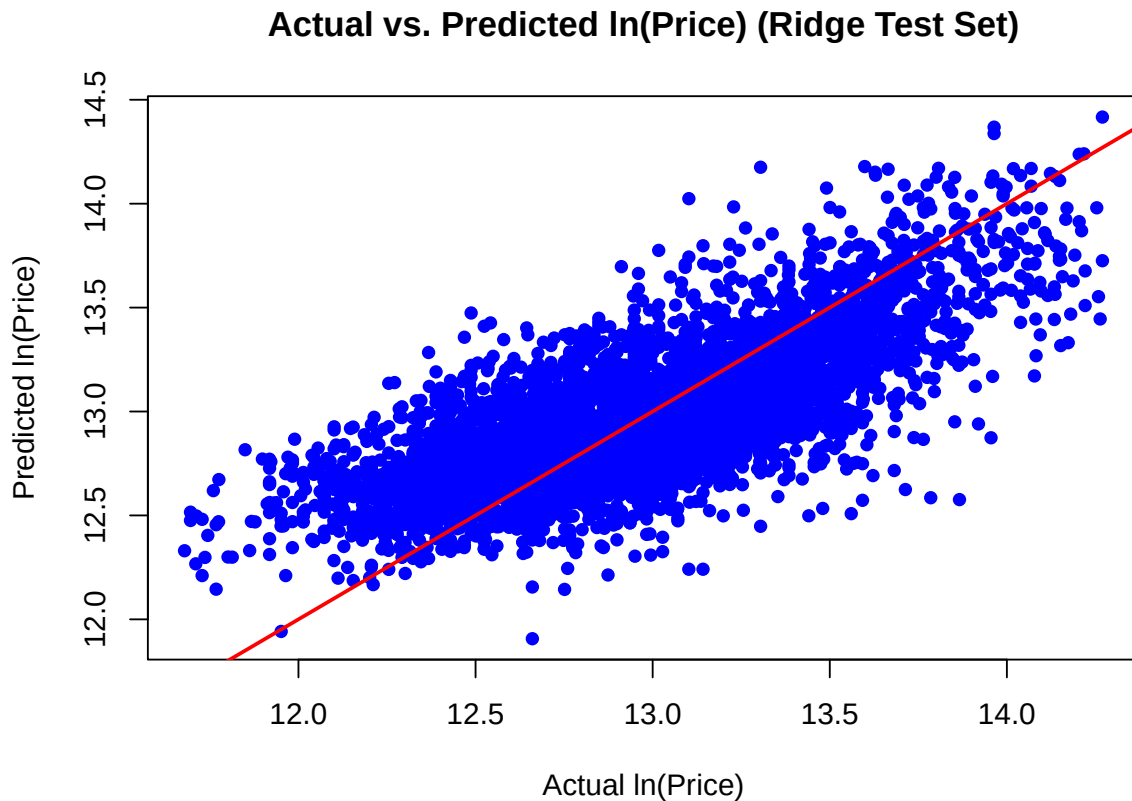
```
print(ridge_df)
```

```
##      Feature Coefficient
## 1 (Intercept) 12.97663043
## 2   bedrooms -0.03672733
## 3   bathrooms  0.08198659
## 4 sqft_living  0.14675825
## 5   sqft_lot -0.02748129
## 6      view   0.02431945
## 7   condition  0.01749271
## 8      grade  0.23267842
## 9   yr_built -0.16990722
```

The Ridge regression model coefficients are shown above. **Note that these coefficients apply to the centered and scaled predictor variables.** They indicate the change in $\ln_price$ for a one standard deviation change in the respective predictor, holding others constant. `sqft_living` and `grade` have the largest positive coefficients, indicating strong influence. Ridge shrinks coefficients towards zero but typically doesn't eliminate them entirely, helping to manage multicollinearity and stabilize the model.

Model Visual of Ridge Regression

```
# Plot Actual vs Predicted ln(price) on the Test set
par(mar = c(5, 5, 4, 2) + 0.1)
plot(Y_test_reg, ridge_preds_test, main = "Actual vs. Predicted ln(Price) (Ridge Test
↪ Set)",
     xlab = "Actual ln(Price)", ylab = "Predicted ln(Price)", col = "blue", pch = 16)
abline(0, 1, col = "red", lwd = 2) # Add y=x line
```



The graph shows a reasonably high positive relationship between actual and predicted house prices, which means that the Ridge Regression model is a good fit for this data. The points cluster relatively close to the diagonal line, which points towards good predictions with somewhat minimal error.

```
# Train Lasso CV on training data
set.seed(1)
lasso_cv <- cv.glmnet(X_train_mat, Y_train_reg, alpha = 1, lambda = lambdas, nfolds = 10)
best_lambda_lasso <- lasso_cv$lambda.min
cat("Lasso best lambda:", best_lambda_lasso, "\n")
```

Lasso Regression Evaluation

```
## Lasso best lambda: 0.001
```

```
# Fit final Lasso model on training data using best lambda
lasso_model <- glmnet(X_train_mat, Y_train_reg, alpha = 1, lambda = best_lambda_lasso)

# Predict on the scaled test data
lasso_preds_test <- predict(lasso_model, newx = X_test_mat)

# Evaluate on the test set using the defined function
lasso_test_results <- eval_metrics(Y_test_reg, lasso_preds_test) # Use original
↪ Y_test_reg
```

```
print(paste("Lasso Regression - Test RMSE:",
            round(lasso_test_results$RMSE, 5), "Test R²:", round(lasso_test_results$R2,
            ↪ 5)))
```

```
## [1] "Lasso Regression - Test RMSE: 0.30616 Test R²: 0.54371"
```

```
# Display coefficients from the Lasso model (these are based on SCALED data)
lasso_coeffs <- coef(lasso_model)
lasso_df <- data.frame(Feature = rownames(lasso_coeffs), Coefficient =
  ↪ as.vector(lasso_coeffs))
lasso_df <- lasso_df[lasso_df$Coefficient != 0, ] # Show non-zero coefficients

cat("Lasso Coefficients (on Scaled Data):\n")
```

Lasso Coefficients

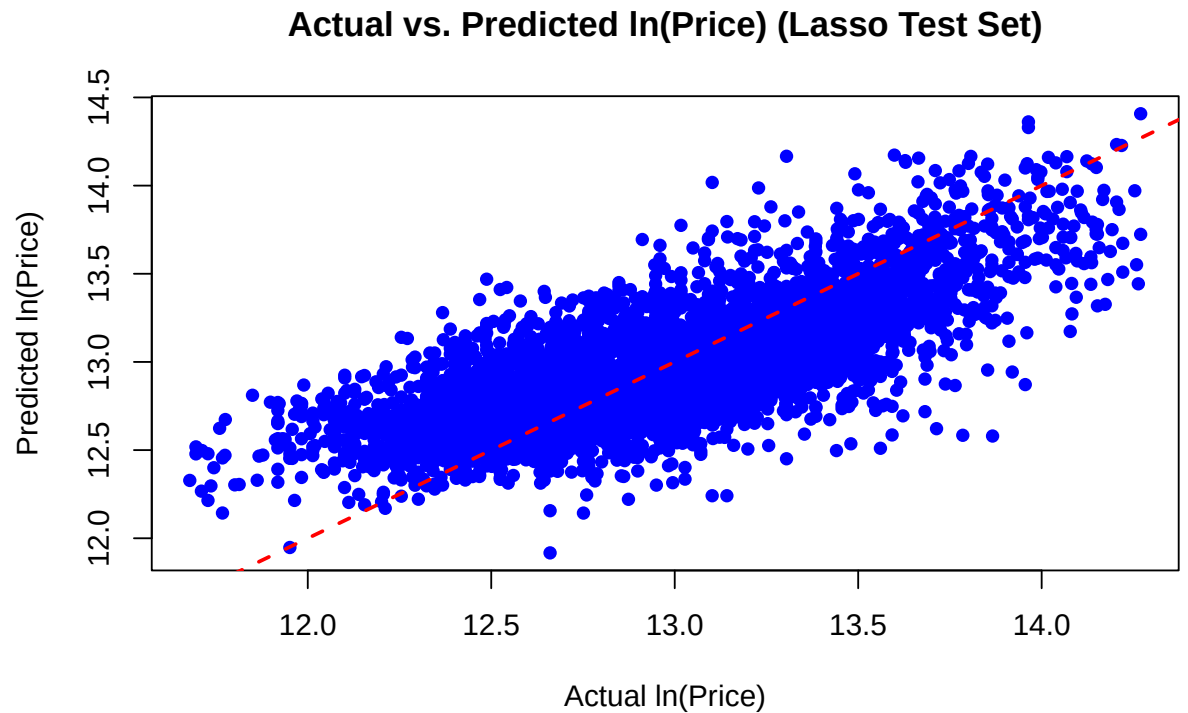
```
## Lasso Coefficients (on Scaled Data):
```

```
print(lasso_df)
```

```
##      Feature Coefficient
## 1 (Intercept) 12.97663043
## 2   bedrooms -0.03417933
## 3   bathrooms  0.08006553
## 4 sqft_living  0.14493904
## 5   sqft_lot -0.02626226
## 6      view   0.02367990
## 7   condition  0.01662926
## 8      grade  0.23283953
## 9   yr_built -0.16845539
```

The Lasso regression model coefficients are shown above. **Note that these coefficients apply to the centered and scaled predictor variables.** Lasso performs feature selection by potentially shrinking some coefficients to exactly zero. In this case, all predictors remained in the model at the chosen lambda, suggesting they all contribute. Similar to Ridge, sqft_living and grade show the strongest positive influence on the scaled data.

```
# Plot Actual vs Predicted ln(price) on the Test set
plot(Y_test_reg, lasso_preds_test, main = "Actual vs. Predicted ln(Price) (Lasso Test
  ↪ Set)",
     xlab = "Actual ln(Price)", ylab = "Predicted ln(Price)", col = "blue", pch = 16)
abline(0, 1, col = "red", lwd = 2, lty = 2) # Add y=x line
```

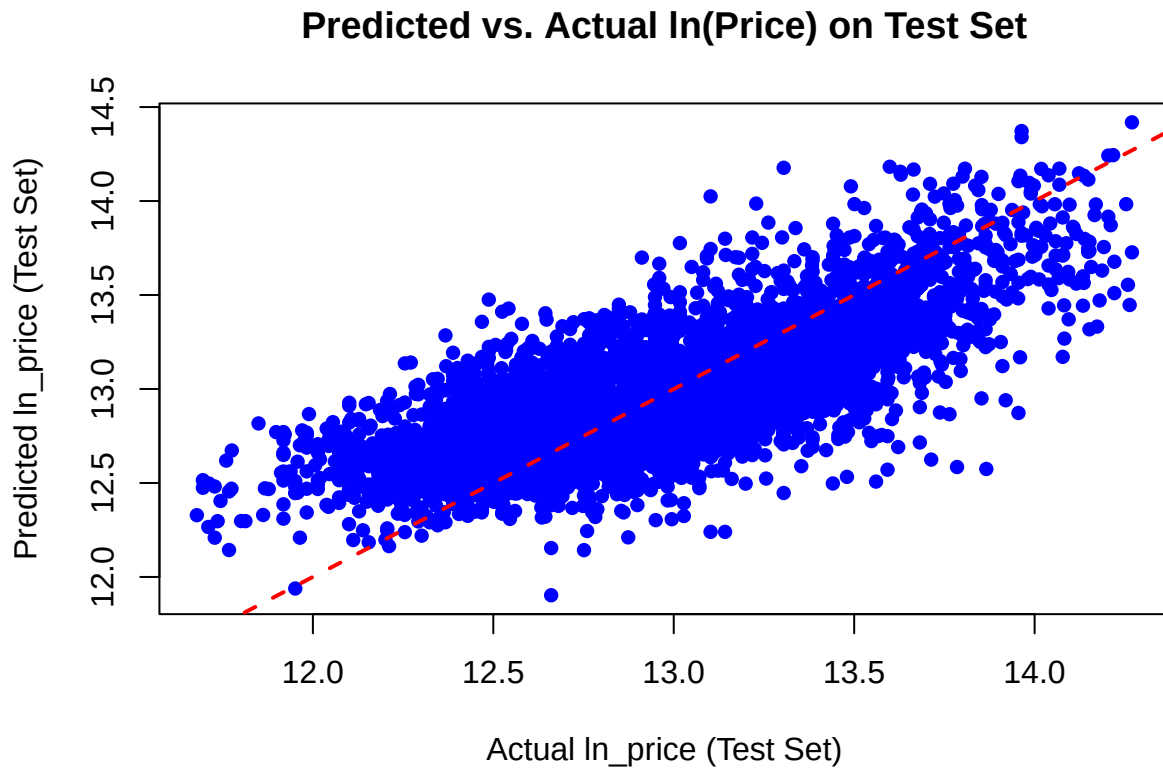


Lasso Model

The graph shows a reasonably strong positive relationship between actual and predicted log-prices on the test set for the Lasso model, very similar to Ridge.

Prediction of ln(Price)

```
plot(actual_test, predictions_test,  
      xlab = "Actual ln_price (Test Set)",  
      ylab = "Predicted ln_price (Test Set)",  
      main = "Predicted vs. Actual ln(Price) on Test Set",  
      col = "blue", pch = 16)  
abline(a = 0, b = 1, col = "red", lwd = 2, lty = 2)
```



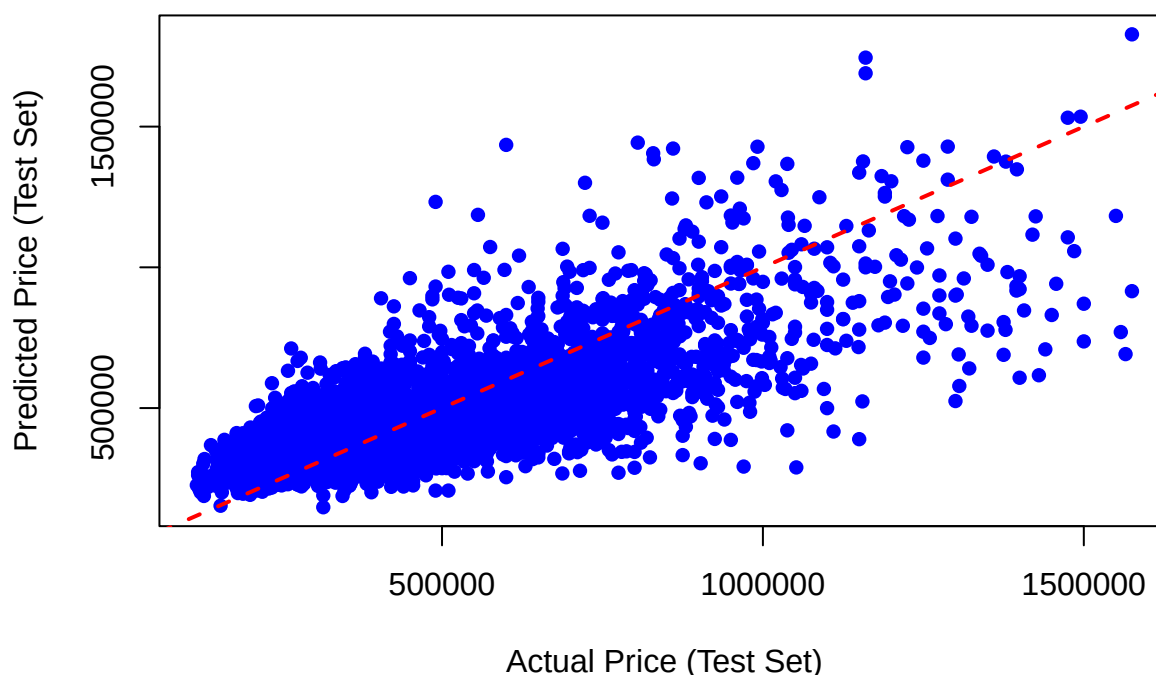
We will proceed to model out the price based predictions in the data as seen below

Prediction of Price

```
predicted_prices_test <- exp(predictions_test)
actual_prices_test <- exp(actual_test) # Or use test_data$price

plot(actual_prices_test, predicted_prices_test,
      xlab = "Actual Price (Test Set)",
      ylab = "Predicted Price (Test Set)",
      main = "Predicted vs. Actual Prices on Test Set",
      col = "blue", pch = 16)
abline(a = 0, b = 1, col = "red", lwd = 2, lty = 2)
```

Predicted vs. Actual Prices on Test Set



The points are clustered around the $y = x$ identity line, meaning that the predicted price from our model is usually close to the true price. This would suggest the model is fitting relatively well to the data.

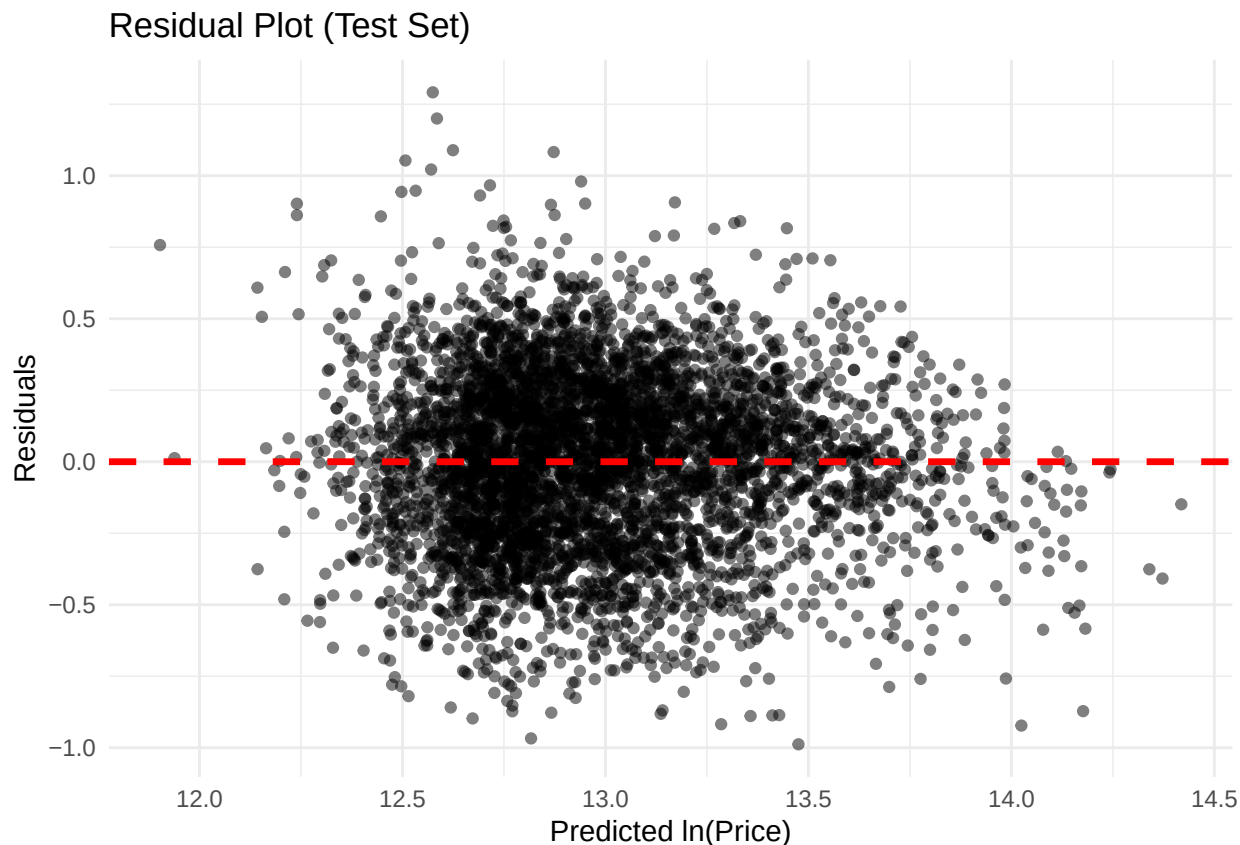
Residuals

```
residuals_test_vals <- actual_test - predictions_test # Use test set values

residuals_df_test <- tibble(
  predictions = predictions_test,
  residuals = residuals_test_vals
)

ggplot(residuals_df_test, aes(x = predictions, y = residuals)) +
  geom_point(alpha=0.5) +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed", size = 1.2) +
  labs(title = "Residual Plot (Test Set)", x = "Predicted ln(Price)", y = "Residuals") +
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



The Residuals plot shows the model to be homoscedastic. The residuals appear to be roughly even distributed throughout the model showcasing that the variance is roughly equal.

Conclusion and Summary

For this project, we created a multivariable regression model, which was designed to predict house prices in Seattle. Our data came from the dataset 'House Sales in King County, USA', Seattle being in King County. The dataset and our subsequent analysis involved various explanatory variables, including square footage, grade(quality), number of bathrooms, condition, view quality, and year built.

We began our analysis by performing backwards stepwise regression using stepAIC, resulting in a stepwise model with many predictors, which would be used to predict the log transform of price, which was used instead of just price to improve distribution symmetry. We then applied Cook's Distance to identify influential points in our data, and we removed them. We also removed the predictor sqft_above in order to address multicollinearity, and we removed the predictor floors due to its limited significance and to remove complexity from the model.

After some refinement, we were able to obtain a model with 6 predictors with a moderate adjusted R^2 value of 0.6155, and an okay RMSE of 0.2659. We then evaluated the model using the anova() function, to get Sum Squares, Mean Squares, p values, and F values, which generally showed that the variables chosen in the model, being sqft_living, grade, bathrooms, view, condition, and yr_built, effectively accounted for most of the variability in ln(price).

Best subset selection was then used to select the predictors providing the highest adjusted R^2 . Ridge Regression was used, and added stability by properly weighting different features, while also reducing multicollinearity among correlated variables. Lasso Regression was used, and similarly reduced the impact of less important features in the model, creating a more stable predictive model.

Normalization and Alternative regression based approaches are likely useful tools in improving overall model quality. Looking at the positives of our project, our model ended up demonstrating some predictive ability, with an adjusted R squared of well over 0.6. Clearly, home price can be, to some degree, explained by our predictors. Also, we were able to symmetrize our response variable with a log transform, which better prepares our data for linear regression.

We were also able to simplify our model significantly via removing insignificant/problematic variables, while still maintaining predictive ability. Looking at the negative aspects of our model, we were unable to include certain key variables relating to socioeconomic factors that could affect house price, including safety, school/park access, quality of schools...etc, and so we missed out on including factors that could improve predictive ability.

Also, our improved regression model still showed tailing on both ends of our Q-Q plot. In the future, we could include these socioeconomic factors, along with location based factors that could identify certain areas that are more undesirable to live in than others. Also, it would be useful from a practical standpoint to have more recent information, considering that this model is meant to help better understand the real life issue of home pricing. In particular, factors like Neighborhood, Crime Rate, Public Transport Access, Commute Time, Natural Disaster Risk, and Pollution could all improve model accuracy.

References

- 1) The dataset used in this study is the “House Sales in King County, USA” dataset, obtained from <https://www.openml.org/search?type=data&status=active&id=42635&sort=runs>. The Author of the Dataset can be found at <https://www.kaggle.com/harlfoxem/>.
- 2) Article entitled “Multiple Linear Regression in R” from sthda.com. This article can be found at <https://www.sthda.com/english/articles/40-regression-analysis/168-multiple-linear-regression-in-r/>, and was written by user “kassambara”.
- 3) Malpezzi, Stephen. “Housing Prices, Externalities, and Regulation in U.S. Metropolitan Areas.” *Journal of Housing Research* 7, no. 2 (1996): 209–41. <http://www.jstor.org/stable/24832860>.
- 4) Yadavalli, Anita, Brenna Rivett, James Brooks, and Christiana K. McFarland. “A Comprehensive Look at Housing Market Conditions Across America’s Cities.” *Cityscape* 22, no. 2 (2020): 111–32. <https://www.jstor.org/stable/26926899>.